

Practica 3

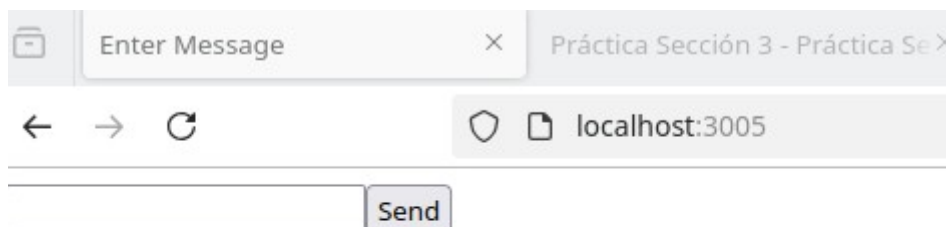
1. Prueba de arranque: Ejecuta node app.js (o el nombre de tu archivo). La consola no debe arrojar errores y el proceso debe mantenerse en ejecución escuchando en el puerto 3000.

```
Practica 3 > node app.js > ...
1  const http = require('http');
2
3  const routes = require('./routes');
4
5  //console.log(routes.someText);
6
7  //Tarea 1
8  const server = http.createServer(routes.handler);
9
10 server.listen(3005);
```

Problems Output Debug Console Terminal Ports node - Practica 3

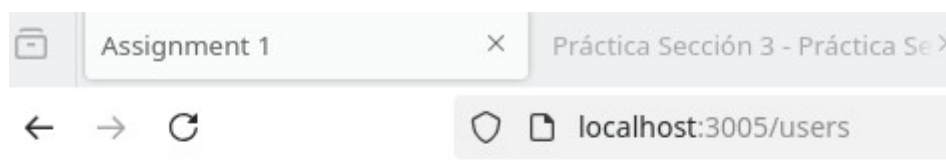
```
alumnado@S09-PC23:~/Descargas/DAW-2026/DWES/UD3 Servidor Web/Practica 3$ ls
app.js routes.js
alumnado@S09-PC23:~/Descargas/DAW-2026/DWES/UD3 Servidor Web/Practica 3$ node app.js
```

2. Prueba de /: Abre un navegador en <http://localhost:3000/>. Debes ver el formulario con el campo de texto y el botón "Send".



3. Prueba de /users: Accede a <http://localhost:3000/users>. Debe mostrar la lista con "User 1" y "User 2".

Desarrollo
web en
entorno
servidor -
2ºDAW

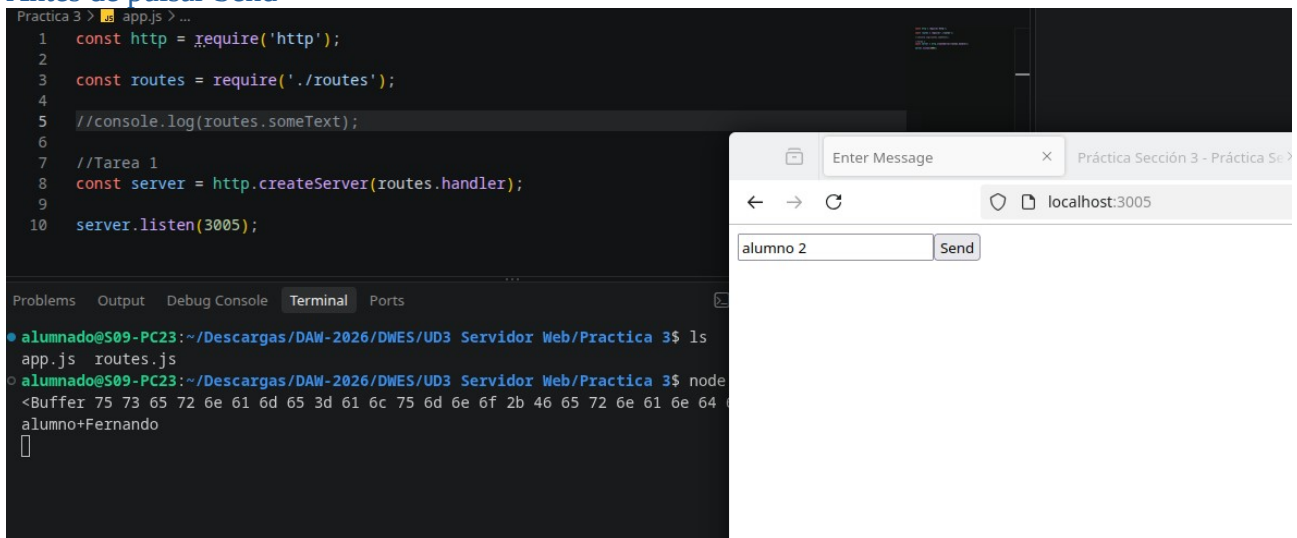


- User 1
- User 2

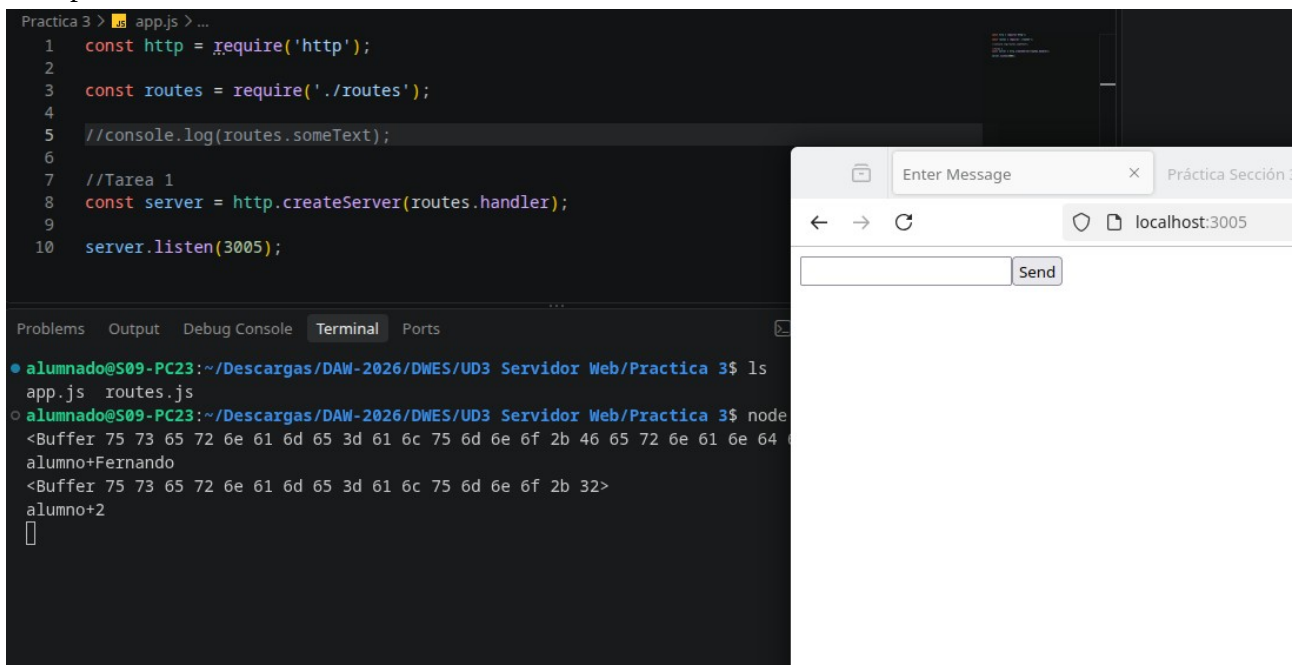
4. Prueba de envío POST:

- Vuelve a <http://localhost:3000/>, escribe un nombre (por ejemplo, alumno1) y pulsa "Send".
- En la terminal donde corre Node.js debe imprimirse alumno1.
- El navegador debe volver a cargar automáticamente la página principal <http://localhost:3000/>

Antes de pulsar Send



Tras pulsar Send



Reto: Guardar lista en users.txt

Objetivo: Genera un nuevo js que además pueda guardar los usuarios del formulario en un archivo de texto (users.txt) y mostrar la lista real de forma dinámica en /users.

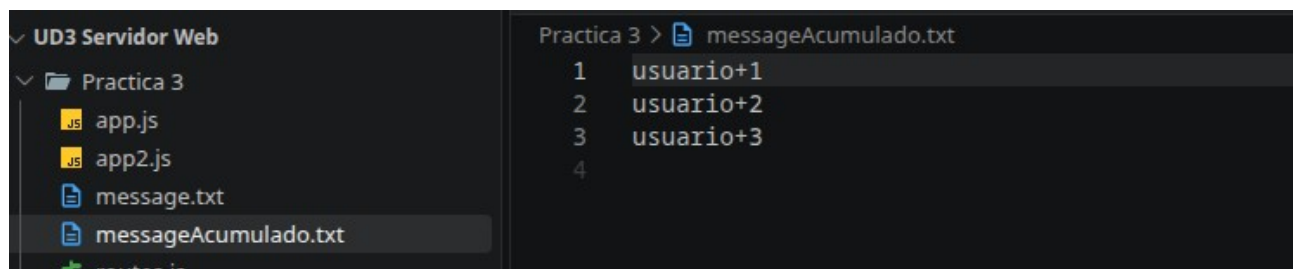
Requisitos

1. Guardar usuario (/create-user): Al recibir el formulario POST, añade el nombre a users.txt sin borrar los anteriores usando fs.appendFile(). La redirección 302 a / debe ir dentro del callback de escritura.

La programación de appendFile

```
req.on('end', () => {  
  const parsedBody = Buffer.concat(body).toString();  
  const message = parsedBody.split('=')[1];  
  fs.appendFile('messageAcumulado.txt', message + '\n', err => {  
    res.statusCode = 302;  
    res.setHeader('Location', '/');  
    return res.end();  
  });  
});
```

El resultado:



```
Practica 3 > messageAcumulado.txt  
1 usuario+1  
2 usuario+2  
3 usuario+3  
4
```

2. Mostrar usuarios (/users): Lee users.txt de forma asíncrona (fs.readFile()) y genera dinámicamente las etiquetas . Si no hay usuarios, muestra un mensaje por defecto.

3. No bloqueante: Prohibido usar métodos síncronos (writeFileSync / readFileSync)

el código

```

if (url === '/users'){

    //lectura asincrona de messageAcumulado, se recibe err (si error) o el valor leído
    fs.readFile('messageAcumulado.txt', 'utf-8', (err, lectura) => {
        if (err) {
            console.log(err);
            return res.end('Error al leer el archivo'); //si error mensaje y fin
        }

        const datos = lectura.split('\n'); //datos leídos string pasamos a Array cortando

        res.write('<html>');
        res.write('<head><title>Assignment 1</title></head>');
        res.write('<body>');

        datos.forEach((dato) => { //leemos array datos y para cada uno vemos si está vacío
            if (dato.trim() !== '') {
                res.write('<li>${dato}</li>');
            }
        });

        res.write('</body>');
        res.write('</html>');

        return res.end();
    });

    return;
};

```

El resultado:

